Strata — Hướng dẫn BA

Dành cho Business Analyst

strata-kb v1.18.0 · 2026-09-19

1. Tài liệu này dành cho ai

Bạn viết yêu cầu. Ticket, user story, tiêu chí chấp nhận — và với những tính năng lớn thì có cả mission plan để chia nhỏ chúng ra.

Vấn đề của bạn không phải là viết. Vấn đề là **bám nguồn**: làm sao để khi ticket ghi "trường này dài 4 ký tự, chữ và số", khẳng định đó đến từ một tài liệu có thật, ở một phiên bản xác định, và vẫn kiểm chứng được vài tháng sau khi có người hỏi nó lấy ở đâu ra.

Strata cho bạn ba thứ để làm việc đó:

1. **Tìm kiếm trên tri thức đã publish**, để bạn tìm được đoạn cần thay vì phải nhớ.
2. **Trích dẫn ghim phiên bản**, để ticket ghi lại đúng những gì tài liệu nói vào ngày bạn viết.
3. **Cổng Definition of Ready**, để một ticket không thể đến tay lập trình viên với trích dẫn hỏng hoặc thiếu.

Và khi mission và ticket của bạn đã sẵn sàng, bạn có thể **publish plan** lên hub, để lập trình viên và các team khác tìm thấy *cái gì đang được lên kế hoạch* theo cùng cách họ tìm cái đã có (mục 5), và **sửa đổi** mission hay ticket tại chỗ khi tri thức, architecture, phạm vi hay quá trình giao hàng dịch chuyển bên dưới nó (mục 6).

1.1 Repo BA làm gì và không làm gì

Có làm	Không làm
Đọc tri thức đã publish trên hub	Ingest tài liệu
Trích dẫn tri thức đó, ghim theo commit của hub	Tóm tắt bất cứ thứ gì
Quản lý phiên bản ticket và mission trong Git	Publish nội dung tri thức
Kiểm soát chúng bằng CI	Đẩy lên hệ thống quản lý issue
Publish plan của chính nó (`<repo_id>-plan`) theo yêu cầu	Publish bản nháp, hay bất cứ thứ gì bạn chưa đánh dấu là sẵn sàng

Repository của bạn hoàn toàn **chỉ đọc** đối với cơ sở *tri thức*. Bạn trích dẫn, bạn không bao giờ đóng góp. Điều này là có chủ đích: nó giữ cho yêu cầu nghiệp vụ truy vết được tới một phiên bản tri thức mà không trao cho quy trình làm yêu cầu quyền thay đổi chính tri thức đó. Thứ duy nhất bạn gửi lên hub là plan của chính bạn: các mission và ticket đã review, dưới dạng một tài liệu do máy sinh. Nó nằm cạnh tri thức; nó không bao giờ sửa tri thức.

Trợ lý cũng không bao giờ tự mở ticket thay bạn. Nó tạo ra Markdown; bạn đọc, commit, và tự dán vào hệ thống quản lý issue.

2. Thiết lập một lần

```
pip install strata-kb
kb init --kind ba
```

2.1 Trỏ tới hub

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
repo_id: acme-requirements
intake: ""   # đặt giá trị này để publish plan, xem 2.5
```

`hub:` là nơi `kb ticket lint` và `kb query` phân giải tham chiếu. `repo_id` không còn chỉ là một nhãn: nó đặt tên cho tài liệu bạn publish, `<repo_id>-plan`, và phải khớp với mục của repo bạn trong registry của hub (2.5).

2.2 Nối trợ lý của bạn với hub

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) đã được scaffold sẵn, nối tới các công cụ tìm kiếm và trích dẫn của hub — nhưng qua hai placeholder, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`, mà chưa ai điền giá trị.

Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Lệnh này hỏi URL HTTP cơ sở và token của hub — xin token từ người quản trị hub — ghi cả hai vào `.env`, đảm bảo git bỏ qua file đó, rồi xác minh chúng với hub, để một URL sai và một token bị từ chối trả về hai lỗi khác nhau:

Biến	Ví dụ
`STRATA_KB_HUB_URL`	`http://kb-hub.example.com:8321`
`STRATA_KB_HTTP_TOKEN`	bearer token của hub

Chạy lại `kb mcp-setup` một lần nữa (không kèm gì) sẽ đọc lại cả hai giá trị từ `.env` và xác minh lần nữa mà không cần nhập lại token. Nên dùng prompt ẩn hoặc biến môi trường `STRATA_KB_HTTP_TOKEN` thay vì cờ `--token` — cờ đó để lại token trong lịch sử shell của bạn.

`.mcp.json` và `.cursor/mcp.json` đọc hai biến đó từ môi trường tiến trình, không đọc trực tiếp từ `.env`, nên hãy nạp `.env` vào shell của bạn (`set -a; source .env; set +a`, hoặc dùng direnv) rồi khởi động lại trợ lý — MCP chỉ đọc môi trường lúc khởi động.

2.3 Mở repository trong trợ lý

`ba-ticket-author` và `ba-mission-plan` được dựng sẵn dưới dạng skill, command và prompt cho **Claude Code, GitHub Copilot và Cursor**. Bạn dùng cái nào thì pipeline cũng như nhau.

2.4 Cấu hình cổng CI

Thiết lập	Ở đâu	Là gì
`STRATA_KB_HUB`	Actions → **Variables**	Hub mà cổng dùng để phân giải tham chiếu
`KB_HUB_TOKEN`	Actions → **Secrets**	Token đọc — chỉ cần với hub riêng tư
`KB_FAIL_ON_STALE`	**Variables**, tuỳ chọn	Đặt giá trị bất kỳ để một sửa đổi phía trên làm cổng fail

Sau đó, trong branch protection, hãy **yêu cầu check `kb-ticket-lint`** trên nhánh mà ticket của bạn đổ về. Việc lint chạy trong CI tự nó không chặn merge; việc yêu cầu check mới chặn.

> Pull request mở **từ một fork** không đọc được secret của repository, nên với hub riêng tư cổng sẽ fail ở đó kèm thông báo không kết nối được hub. Đó là cổng từ chối báo xanh khi chưa kiểm tra được, không phải lỗi mạng. Hãy merge đóng góp từ fork qua một nhánh trong chính repository này.

2.5 Bật publish plan (tuỳ chọn, một lần cho mỗi repo)

Publish cần hai thứ, mỗi bên một việc:

1. **Người vận hành hub đăng ký repo của bạn** trong `federation/registry.yaml` của hub: khoá là `owner/repo` của bạn, ánh xạ tới `repo_id` của bạn. Họ để trống `workflow:`, vì intake so nó với toàn bộ `job_workflow_ref`, trong đó có tag publish, nên một ghim sẽ không bao giờ khớp.
2. **Bạn đặt `intake:`** trong `.kb/config.yaml` thành URL của dịch vụ intake của hub (người vận hành đưa cho bạn). Repo tạo trước tính năng này không có key `intake:`: hãy thêm `intake: <url>` bằng tay, rồi chạy lại `kb init --kind ba`. Lệnh đó ghi workflow `kb-publish`, skill `kb-publish` (cùng các file bọc cho Copilot và Cursor) và `.gitattributes`, và thêm `.kb/*-plan/` và `.kb/index.yaml` vào `.gitignore`. Nó không đụng tới `config.yaml` của bạn.

Cho tới khi đặt `intake:`, `kb publish` đi đường snapshot (mirror trực tiếp hoặc PR trên hub, không qua CI hay intake). Cả `.kb/<repo_id>-plan/` lẫn `.kb/index.yaml` đều do `kb plan-ingest` sinh ra nên được gitignore: đừng bao giờ commit chúng.

---

3. Viết một ticket

Gọi `/ba-ticket-author` và mô tả nhu cầu nghiệp vụ. Agent chạy một pipeline chín bước; phần việc của bạn là bước 1, 3 và 9.

```
 1 Intake             bạn mô tả nhu cầu
2 Parent mission    tuỳ chọn — nêu mission mà story này thuộc về
3 Ground            kb_search; CHÍNH BẠN chọn section nào áp dụng
4 Draft             story, AC dạng Given/When/Then, use case, business flow
5 Pin               kb_context_new nhúng khối đã ghim
6 Lint              kb ticket lint cho tới khi DoR: PASS
7 Ground technical  SA điền Technical grounding; kb ticket check
8 Maturity review   hai lượt review độc lập, mặc định một vòng
9 Review → save     bạn đọc, commit, dán vào hệ thống issue
```

3.1 Intake

Hãy mô tả nhu cầu bằng ngôn ngữ nghiệp vụ: năng lực cần có, ai cần, và giá trị mang lại. Đừng vội viết giải pháp. Các bước sau của agent hoạt động tốt hơn khi xuất phát từ một vấn đề rõ ràng, thay vì từ một cách hiện thực đã chốt sẵn.

3.2 Bám nguồn — bước thực sự quan trọng

Agent gọi `kb_search` và trình cho bạn **mọi section ứng viên**, không chỉ cái nó thích nhất. Hãy đọc. Hãy chọn những cái thực sự áp dụng được.

Đây là bước duy nhất mà phán đoán nghiệp vụ của bạn không uỷ quyền được. Agent tìm được các đoạn ứng viên; nó không thể biết rằng §4.12 trên thực tế đã bị một chính sách nội bộ thay thế, hay hai section trông giống nhau lại khác nhau ở chỗ quan trọng trong tình huống này. Mọi thứ phía sau — bản nháp, việc ghim, phần hiện thực của lập trình viên — đều đứng trên lựa chọn này.

3.3 Soạn nháp

Agent điền các mục **cốt lõi** của template: tóm tắt, user story, bối cảnh, tiêu chí chấp nhận, use case, một sơ đồ business flow, dependencies, out of scope, open questions. Mỗi tiêu chí chấp nhận là một Given/When/Then — một trạng thái ban đầu, một hành động, một kết quả quan sát được — và mọi khẳng định chạm tới một tiêu chuẩn đều mang trích dẫn `[doc-id §section]`.

Các mục **mở rộng** chỉ có khi story cần: sequence diagram khi tương tác trải qua nhiều hơn một hệ thống (tích hợp, callback), non-functional requirements khi story chạm tới tải hay thời gian, UI spec khi có màn hình, test data khi xử lý bản ghi hay tập tin. Còn lại thì xoá. Lint cảnh báo khi chính lời story gọi tên một mục đang thiếu; bạn trả lời bằng mục đó hoặc `N/A — <lý do>`. Sequence ở mức code — service, bảng, message — là việc lập trình viên vẽ lúc thiết kế, không phải việc BA đoán.

3.4 Ghim

Khi bạn xác nhận các section, agent gọi `kb_context_new` và nhúng khối `## KB context` trả về. Khối đó ghi lại commit của hub tại thời điểm này. Từ đây ticket không còn nói "xem sổ tay" — nó nói "đây chính xác là điều sổ tay nói vào ngày viết ticket này, và đây là cách kiểm chứng".

3.5 Lint

`kb ticket lint` chạy, agent sửa những gì nó báo, và lặp lại cho tới khi `DoR: PASS`.

3.6 Maturity review

Khi lint đã pass, agent chạy hai lượt review độc lập dựa trên `docs/review-rubric.md` — một lượt chấm **Business coverage**, một lượt chấm **Dev implementability**. Nó áp dụng sửa đổi, lint lại, và ghi vòng đó. Với một ticket, mặc định đó là toàn bộ phần review: những gì còn mở được bàn giao cho bạn dưới dạng câu hỏi có người phụ trách, điểm dưới 4 được báo, không bị đuổi theo. Muốn thêm vòng thì yêu cầu; ticket có hơn sáu tiêu chí chấp nhận tự động được thêm, tối đa ba vòng hoặc tới khi cả hai trục đạt 4.

Khoảng trống nào nó không tự khép được sẽ trở thành một câu hỏi mở có người phụ trách, viết là `OPEN(<owner>)`, thay vì một phỏng đoán. Kết quả nằm ở mục `## Review record` của ticket.

Hãy đọc các câu hỏi mở. Đó là chỗ agent nói cho bạn biết nó biết là nó chưa biết.

3.7 Bạn review

Bản nháp được lưu vào `tickets/<folder>/<ticket-id>.md` (folder là mission cha, hoặc một slug epic hay tính năng bạn đặt tên). Bạn đọc, commit, rồi dán vào hệ thống quản lý issue. Trợ lý không bao giờ làm việc đó thay bạn. Thứ bạn dán là `kb ticket export tickets/<folder>/<ticket-id>.md`: cùng ticket đó nhưng không còn các chú thích hướng dẫn của template và hai mục nội bộ của BA, `## Definition of Ready` và `## Review record`. `## KB context`, `## Dependencies` và `## Technical grounding` được giữ — lập trình viên đọc chúng.

3.8 SA ghim phần kỹ thuật

Bước 7 làm việc này thay bạn. Khi lint báo `DoR: PASS`, agent lưu bản nháp rồi gọi `/sa-ticket-ground` trên chính file đó như một subagent riêng. Lệnh này điền mục `## Technical grounding` do SA sở hữu — service, file, bảng,

route, external, volumes, lệnh healthcheck và device reservations, lệnh test — mỗi dòng là một section id lấy từ tài liệu `<repo>-code` trên hub. Nó không bao giờ sửa mục của bạn; một phát biểu nghiệp vụ mâu thuẫn với sự thật trong code sẽ được trích lại ở đó, không bị sửa.

Code chưa tồn tại không phải là thiếu dữ liệu, mà là một quyết định thiết kế. SA đề xuất nó thành một dòng trong `## Technology decisions` của mission cha (trạng thái `OPEN`, chủ sở hữu là một con người) và ghi `[NEW: D<n>]` trỏ tới dòng đó. Chỉ những gì code đã có mà tài liệu không chứng minh được — luồng nội bộ, tình huống lỗi, thân request — mới được gác dưới `Open decisions` cho lập trình viên.

`kb ticket check tickets/<ticket-id>.md` là cổng: PASS chỉ khi mọi id phân giải được, mọi `[NEW: D<n>]` trỏ tới một dòng `DECIDED`, và `Open decisions` rỗng. Một ticket không có mission cha thì không có bảng để giữ dòng, nên SA viết văn bản tự do `[NEW: <reason>]` thay vào đó — gate chấp nhận nó như một ghi chú; một mission sẽ cho quyết định đó một chủ sở hữu. Một dòng `OPEN` làm fail cổng và nêu tên chủ sở hữu — đổi nó sang `DECIDED` là quyết định của bạn, không bao giờ của agent. Bản bàn giao mang theo khối `## Needs input` liệt kê đúng những dòng đó.

Cổng này cũng so revision ở `Grounded on:` với `-code` hiện tại. Revision đó đổi theo mỗi commit của dev, nên chỉ lệch revision thì không bao giờ fail:

- mọi fact được trích vẫn đúng và không section nào được trích thay đổi → PASS với `[note] grounded @<old>, still valid at @<new> — cited sections unchanged`;
- một section được trích có đổi nhưng mọi tên được trích trong đó vẫn còn → PASS với một `[warn]` kèm unified diff của section đó, để lập trình viên tự đánh giá;
- một fact được trích đã mất (id, cột, route, lệnh, đường dẫn `Files:`, compose fact) → FAIL, kết thúc bằng `grounding drifted since @<old> (now @<new>) and no longer holds — re-run /sa-ticket-ground`.

Open decisions và các dòng `[NEW: D<k>]` chưa quyết vẫn fail như trước, không có dòng đó: chúng là quyết định của bạn, không phải việc ground lại.

Chỉ chạy `/sa-ticket-ground` bằng tay khi `kb ticket check` fail với `re-run /sa-ticket-ground`.

---

4. Mission plan — cho tính năng lớn

Một tính năng trải dài nhiều user story thì làm **mission plan** trước. Việc nhỏ thì đi thẳng tới ticket. Mission không bao giờ là bắt buộc.

```
/ba-mission-plan
```

Pipeline là **Intake → Ground → Draft → Split → Ground services → Pin → Lint → Maturity review → Review**, và lưu ra `missions/M-<slug>.md`.

Một mission mang theo:

- Sơ đồ **C4 Level 1** (System Context) và sơ đồ **C4 Level 2** (Container).
- **Phân định phạm vi** — cái gì nằm trong, cái gì rõ ràng nằm ngoài.
- **Backlog user story** có id suy ra từ id của mission.

```
kb mission lint missions/M-checkout-revamp.md
```

Cổng kiểm tra: cấu trúc bắt buộc, có đủ cả hai sơ đồ, backlog đúng dạng với id suy ra từ id mission, và mọi trích dẫn phân giải được ở phiên bản hub đã ghim. Cảnh báo `0/N US drafted` là bình thường — các ticket chưa tồn tại.

Cái vẫn thuộc **phán đoán của bạn**: backlog đã thực sự đầy đủ hay chưa.

4.1 Nối ticket với mission

Khi soạn một story từ backlog, hãy nêu mission cha. Agent sẽ:

- đọc `missions/<mission-id>.md` và lấy tiêu đề story từ dòng backlog,
- ghi `> Parent mission: <mission-id> @ rev <N>` trên một dòng riêng ngay dưới tiêu đề ticket, với `<N>` là `> Revision:` hiện tại của mission (phần `@ rev` là tuỳ chọn; vắng mặt nghĩa là rev 1),
- lưu ticket thành `tickets/<mission-id>/<mission-id>-US<n>.md`.

Chính tên file đó là thứ phép kiểm tra liên kết ngược tìm kiếm. Bỏ dòng parent đi vẫn hợp lệ — chỉ là ticket không truy vết được về mission của nó. Ghim này quan trọng khi bạn sửa đổi mission (mục 6.3): nhờ nó `kb mission next` biết ticket được viết theo một revision cũ hơn. `kb ticket tidy` chuyển một `tickets/` phẳng vào bố cục theo từng mission dựa trên dòng đó.

4.2 Đánh số

Backlog có lỗ hổng số thứ tự là chuyện bình thường. Nếu bạn bỏ một story, hãy để số của nó nghỉ hưu. Đánh số lại sẽ làm hỏng tên file của những ticket đã soạn.

4.3 SA điền danh sách service

Bước 5 làm việc này thay bạn. Khi bạn đã xác nhận backlog và `## Sequencing` đã điền, agent gọi `/sa-ticket-ground --mission` và nó điền `## Services & order` — mỗi dòng một `svc.<name>` lấy từ tài liệu `<repo>-code` trên hub, cột `Depends on` chép từ chính record đó. Service mà mission sẽ tạo mới mang `[NEW: D<n>]`, trỏ tới một dòng SA thêm vào `## Technology decisions` để bạn quyết định. Gate sau đó báo FAIL đúng những dòng đó cho tới khi bạn chuyển chúng sang `DECIDED` — đó là kết quả mong đợi, không phải lỗi. Chỉ ở lớp năng lực: không tên file, không bảng, không route — những cái đó thuộc về `## Technical grounding` của từng ticket, điền sau ở mục 3.8.

Ở repo skeleton, mỗi dòng mà tài liệu architecture chứng minh được đều trích dẫn nó ngay trong ô Decision (`[<arch-doc> §<section>]`), và agent dừng ở bước 5b — **Decide** — để trình cả bảng cho bạn một lần. Bạn chuyển những dòng có trích dẫn mà bạn xác nhận sang `DECIDED`; dòng không có trích dẫn giữ `OPEN` kèm chủ sở hữu. Bạn quyết định ở đây, một lần, không phải từng ticket. Story đầu tiên của mission như vậy là **foundation slice**: những gì tài liệu architecture nói phải có trước mọi story tính năng, và mọi story khác phụ thuộc vào nó trong `## Sequencing`.

4.4 Ticket kế tiếp

```
kb mission next
```

Mọi story trong backlog của toàn bộ `missions/` được báo là một trong năm trạng thái:

Trạng thái	Nghĩa là
`done`	Id của nó nằm trong một dòng `hist.*` của `<repo>-svc` trên hub — thứ `kb svc note` ghi lúc lập trình viên bàn giao và CI publish khi merge
`ready`	Đã có file ticket và mọi ô `## Definition of Ready` đã được tick
`draft`	Đã có file ticket nhưng còn ô DoR chưa tick, hoặc không có mục DoR, hoặc mission cha đã đi tiếp (xem dưới)
`to-draft`	Chưa có ticket, mọi story trong `Depends on` đã `done`, mọi dòng D chặn nó đã `DECIDED`
`blocked`	Nêu rõ lý do: `US <id> not done`, `D<n> OPEN (owner: <x>)`

Báo cáo kết thúc bằng `Next: <us-id> — <title>`; `/ba-ticket-author` không kèm tham số sẽ chạy lệnh này trước và đề xuất đúng story đó. Một phụ thuộc mới chỉ `draft` hay `ready` vẫn chặn: done nghĩa là đã merge. Thứ tự giữa các mission được ghi trong ô `Depends on` bằng story của mission khác (`M-<other>-US<n>`).

Một ticket cũng được báo `draft`, với lý do `parent mission rev <M> affects this story`, khi hai điều cùng đúng: `> Revision:` của mission cha cao hơn ghim `@ rev` của ticket, **và** một dòng change log sau ghim đó liệt kê story trong `Affected`. Hãy sửa đổi ticket (mục 6.2) và đặt ghim của nó bằng revision của mission, thì nó lại là `ready` sau khi bạn tick lại.

Lệnh chỉ đọc và thoát 0 sau khi in báo cáo. Hub chưa cấu hình hoặc không kết nối được, `-svc` chưa publish, hay không mission nào có dòng `Grounded on:` đều thành một dòng `note:`, không bao giờ là lỗi — không story nào được đánh `done` chừng nào hub chưa trả lời được.

`kb mission next --from-hub [--repo <ba-repo-id>]` tính cùng báo cáo đó chỉ từ hub: không cần `missions/` hay `tickets/`. Story có ticket đã publish là `ready`; `to-draft` hiện là `not ready (no published ticket)`; các trạng thái chỉ thấy ở repo (`draft`, việc hạ cấp theo revision) không thấy được từ hub, vì những ticket như vậy không được publish. Menu Plans của hub (mục 5.3) hiển thị đúng điều đó trên trình duyệt.

---

5. Publish plan

Khi một mission hoặc ticket đã được review, bạn có thể đưa nó lên hub. Lập trình viên, SA và các team khác khi đó thấy được cái gì đang được lên kế hoạch, không chỉ cái gì đã có. Publish là theo yêu cầu: không có gì tự đi ra khi bạn merge. Phần thiết lập nằm ở mục 2.5.

5.1 Cái gì được publish

`kb plan-ingest` quyết định. Lệnh này mang tính xác định, không cần LLM, không cần hub, và chạy offline. Nó publish:

Mục	Được publish khi
Mission	`kb mission lint` không báo lỗi cấu trúc **và** mọi ô dưới `## Definition of Ready` đã được tick
Ticket	`kb ticket lint` không báo lỗi cấu trúc **và** mọi ô DoR đã được tick **và** `kb mission next` sẽ không hạ nó xuống `draft` (mục 4.4)

Việc một trích dẫn còn phân giải được hay không **không** phải là cổng publish: việc đó cần hub, và `kb-ticket-lint` đã chặn pull request vì nó rồi. Bản nháp không bao giờ được publish. Hai mục `## Review record` và `## Definition of Ready` của bạn cũng không được publish: có mặt trên hub nghĩa là DoR đã được tick đủ.

Mọi thứ còn lại bị bỏ ra ngoài, và mỗi mục in một dòng:

```
skipped: tickets/M-checkout/M-checkout-US3.md — Definition of Ready not fully ticked (6/8)
```

Các lý do bạn có thể gặp:

Lý do	Làm gì
`lint error: <first error>` (`(+N more)` khi có nhiều lỗi)	Chạy `kb mission lint` hoặc `kb ticket lint` trên file đó rồi sửa
`Definition of Ready not fully ticked (<ticked>/<total>)`	Hoàn tất review, rồi tick các ô. Chỉ bạn được tick
`parent mission rev <M> affects this story`	Mission đã đổi sau khi ticket được viết: sửa đổi ticket (mục 6.2)
`fenced line starting with '## ' would split the section — indent it inside the fence`	Một khối code trong file có dòng bắt đầu bằng `##`. Hãy thụt dòng đó vào trong khối code
`id contains whitespace`	Tên file có dấu cách. Đổi tên file
`duplicate id <stem>; kept the first`	Hai file trùng tên. Đổi tên một file
`no '# ' title line`	File không có dòng `# Title`. Thêm một dòng

Một mục bị bỏ qua không bao giờ làm lệnh fail và không chặn publish. Khi chạy trong GitHub Actions, các dòng này cũng được ghi vào step summary. `kb plan-ingest` chỉ thoát `1` khi không chạy được: file đầu vào không đọc được hoặc không phải UTF-8, thiếu `--missions-dir` hay `--tickets-dir`, repo id không hợp lệ, đích đến không do nó sinh ra, `index.yaml` hỏng, hoặc ghi thất bại. Lỗi lint là lý do bỏ qua, không phải lỗi chạy.

5.2 Publish

```
/kb-publish          # hoặc: kb publish
```

`kb publish` chạy `kb plan-ingest` và `kb build` trước, nên thứ bạn xem cục bộ đúng là thứ CI sẽ gửi đi. Hãy tự chạy `kb plan-ingest` trước để thấy mục nào đi, mục nào bị bỏ qua. Sau một lần bạn xác nhận rõ ràng go hoặc no-go, `kb publish` gắn tag `kb-publish/<timestamp>` lên commit rồi đẩy tag. Tag kích hoạt `.github/workflows/kb-publish.yml`, chạy `kb plan-ingest`, `kb build` và `kb ci-publish`; intake của hub sau đó mở một pull request trên hub. Nội dung có hiệu lực khi PR đó được merge.

`kb publish` từ chối nếu `.kb/` còn thay đổi chưa commit. Vì `.kb/<repo_id>-plan/` và `.kb/index.yaml` được gitignore trong repo BA, việc sinh lại chúng không bao giờ làm `.kb/` bẩn. Đừng commit chúng.

Một mục thôi đủ điều kiện (ví dụ bạn bỏ tick một ô DoR khi sửa đổi nó) sẽ rời khỏi plan ở lần publish kế tiếp. Điều này là chủ đích: không để lại placeholder, và lịch sử git của hub giữ mọi phiên bản trước đó.

5.3 Trên hub nó trông thế nào

Plan là một tài liệu duy nhất, `<repo_id>-plan`, nằm cạnh `-code` và `-svc`:

- mỗi mission một **nhóm** (`M-<slug>`): mission, rồi các ticket đã publish của nó;
- một nhóm **`tickets`** cho các ticket không có mission cha đã publish;
- mỗi section là `## <id> <title>`, với `M-<slug>-US<n>` là id của ticket.

Bản L3 là nguồn của bạn đã bỏ dòng tiêu đề, hạ mọi heading xuống một cấp, và bỏ hai mục nội bộ của BA. `## Open questions` được giữ. Bản L2 giữ nguyên từng byte mọi bảng, cộng thêm tóm tắt và mục tiêu nghiệp vụ của mission, hoặc story và tiêu chí chấp nhận của ticket. Agent tìm nó bằng `kb_search`; con người đọc trên web UI ở menu **Plans**, nằm cạnh Documents:

Trang	Hiển thị
`/ui/plans`	Mọi repo BA và các mission của chúng, kèm số lượng theo trạng thái. `?repo=<id>` cho một repo, gồm cả "tickets without mission"
`/ui/plans/<mission>?repo=<id>`	Tóm tắt mission và backlog: trạng thái từng story, lý do, liên kết tới ticket
`/ui/plans/<mission>/<story>?repo=<id>`	Chính ticket đó

Trạng thái được suy ra và chỉ đọc: **done** (story có trong lịch sử `-svc` của repo Dev), **ready** (ticket của nó đã publish), **not ready** (chưa có ticket đã publish, các phụ thuộc đã done) hoặc **blocked** (kèm lý do). Muốn đổi trạng thái thì đổi nguồn: hoàn tất và publish ticket, hoặc để Dev bàn giao. Hệ thống quản lý issue, không phải hub, là nơi cho "đang làm" và người được giao.

Dev chỉ có id ticket thì lấy nó từ đây (`kb ticket fetch <id>`); xem hướng dẫn Developer.

---

6. Giữ mission và ticket luôn cập nhật

Một mission được viết một lần, từ đầu, rồi thực tế dịch chuyển: tri thức được trích dẫn bị sửa, architecture đổi, phạm vi trượt, quá trình giao hàng dạy thêm điều mới. Mục này là cách bạn đổi một mission hay ticket mà không mất dấu lý do.

6.1 Revision và change log

Mỗi mission và ticket mang `> Revision: <N>` ngay dưới tiêu đề, bắt đầu từ 1, và một mục `## Change log` chỉ thêm vào, đặt trước `## Review record`:

```
| Rev | Date | Trigger | What changed | Affected | Review |
|---|---|---|---|---|---|
| 2 | 2026-10-14 | kb-context, scope | Refund window cut to 14 days | M-checkout-US3, M-checkout-US5 | I
```

Cột	Nội dung
`Trigger`	Một hoặc nhiều giá trị `kb-context`, `architecture`, `scope`, `delivery`, cách nhau bằng dấu phẩy
`Affected`	Ở mission: id story **đầy đủ** (`M-<slug>-US<n>`). `US3` trần không khớp, nên ticket sẽ không bao giờ bị gắn cờ. Ở ticket: tên các section. `-` khi không có
`Review`	Điểm maturity của lần sửa đổi, ví dụ `B4 D4 (agent)`. Review của sửa đổi ghi ở đây, không bao giờ ghi vào `## Review record`

Lint cưỡng chế điều này. `Revision` thiếu được đọc là 1 và chỉ cảnh báo, nên file cũ vẫn pass. Khi `Revision: N` lớn hơn 1, log phải có **đúng một dòng cho mỗi revision `2..N`, theo thứ tự**, và mỗi `Trigger` phải là một trong bốn giá trị; nếu không lint fail.

6.2 Sửa đổi

```
/ba-mission-plan --amend M-checkout
/ba-ticket-author --amend M-checkout-US3
```

Không lệnh nào bắt đầu từ Intake, tạo file hay đổi tên gì. Agent:

1. đọc file và các phát hiện của `kb ticket stale --md` cho nó (với mission thì thêm `kb mission next`, để biết story nào đã done);
2. hỏi bạn trigger và điều gì đã đổi. Phát hiện drift điền sẵn câu trả lời, nhưng nó hỏi chứ không đoán;
3. với `kb-context`, ghim lại `## KB context` bằng `kb_context_new` với **toàn bộ** danh sách tham chiếu (giữ nguyên, đã dịch chuyển và mới), thay cả khối. Bạn chỉ xác nhận lại những tham chiếu đã dịch chuyển hoặc mới;
4. với **ticket**, chạy `kb ticket check` ở mọi lần amend. Trigger `architecture` trước hết chuyển cho `sa-ticket-ground`, lệnh này làm mới `## Technical grounding` cùng revision ở `Grounded on:`; mọi trigger khác chỉ gọi SA khi check fail với `re-run /sa-ticket-ground`, và báo cho bạn mọi FAIL khác. Với **mission**, **chỉ** `architecture` mới chuyển cho `sa-ticket-ground --mission` và chạy check: chính các story của mission làm đổi các section `svc.*` nó trích trong lúc giao. Các skill BA vẫn không tự đọc `-code`;
5. áp dụng các quy tắc thay đổi (mục 6.3), tăng `Revision` và thêm dòng change log;
6. lint cho tới khi PASS, và chạy một vòng maturity review **chỉ cho các section đã đổi**;
7. đưa diff cho bạn.

Sửa một ticket cũng **bỏ tick** (các) ô Definition of Ready có bằng chứng đã đổi, và cập nhật ghim `@ rev` khi ticket đã phản ánh mọi dòng change log kể từ ghim cũ có nhắc tới nó. Agent không bao giờ tick ô DoR, không bao giờ commit hay push. Bạn đọc diff, tick lại, commit, rồi `kb publish` để đưa nó đi.

6.3 Quy tắc thay đổi

- **Story đã done thì đóng băng.** Story mà `kb mission next` báo `done` giữ nguyên dòng backlog của nó. Việc mới hoặc việc đã đổi trở thành **một dòng mới với số trống kế tiếp**. Số không bao giờ được dùng lại hay đánh số lại: chúng được trích dẫn trong hệ thống issue, pull request và lịch sử của repo Dev.
- **Story chưa done thì được sửa.** Liệt kê từng cái ở `Affected` của dòng đó.
- **Quyết định được thay thế, không bị sửa.** Một dòng `DECIDED` trong `## Technology decisions` không bao giờ bị viết lại. Thêm một dòng mới `D<n>` với `Supersedes D<k>` trong ô Decision, và đặt trạng thái của `D<k>` thành `SUPERSEDED`. Từ đó `kb ticket check` từ chối mọi `[NEW: D<k>]` vẫn nêu `D<k>`, và cho bạn biết dòng thay thế nó; `kb mission next` xét một blocker `SUPERSEDED` theo dòng thay thế nó, nên story không bị chặn vĩnh viễn. Nếu một mission hay ticket vẫn trích dòng cũ, lần sửa đổi phải mang trigger `architecture`, để SA trỏ lại. Agent liệt kê các file trích dẫn nó.
- **Ticket đi theo mission.** Dòng cha của ticket là `> Parent mission: M-<slug> @ rev <N>`. Khi mission được sửa, mọi ticket có story nằm trong `Affected` của dòng mới sẽ thành `draft` trong `kb mission next` (mục 4.4) và rời khỏi plan đã publish cho tới khi bạn sửa đổi nó.

Publish không cần thêm gì: mục đã sửa đi cùng lần `kb publish` kế tiếp, và `Revision` cùng `## Change log` làm phiên bản đọc được ngay trong tài liệu.

---

7. Dùng tri thức về code

Ngoài tài liệu nghiệp vụ, hub còn giữ hai tài liệu cho mỗi repo sản phẩm, do chính workflow của repo đó publish: `<repo>-code` (cấu trúc do máy sinh — `svc.*`, `db.*`, `api.*`, `int.*`, `cmd.*`, `struct.tree`) và `<repo>-svc` (trách nhiệm do người biên soạn — mỗi service thực sự để làm gì).

Các skill `ba-ticket-author` và `ba-mission-plan` không tự đọc hai tài liệu này. Khi ticket hay mission cần tên service, bảng, route hay file, chúng viết `%%TODO: verify against codebase%%` kèm một câu hỏi mở có chủ, thay vì đoán — rồi `/sa-ticket-ground` (§3.8, §4.3) trả lời những câu đó từ `<repo>-code`, vào đúng mục do SA sở hữu — `## Technical grounding` ở ticket, `## Services & order` ở mission. Mọi id nó viết ra đều được `kb ticket check` đối chiếu với tài liệu; cái tài liệu không chứng minh được — luồng nội bộ, chế độ lỗi, nội dung request — được gác lại dưới `Open decisions` cho lập trình viên, người có code trong tay.

Chính tham chiếu section, viết đúng dạng một trích dẫn thật:

```
<repo>-code §svc.<name>
<repo>-svc §svc.<name>
```

> **Một lưu ý quan trọng.** `<repo>-svc` dùng để bám sơ đồ. Nó không bao giờ thay thế được một trích dẫn nghiệp vụ trong tiêu chí chấp nhận. Một mã, định dạng, enum hay ngưỡng mã hoá một tiêu chuẩn thì vẫn phải đến từ một section nghiệp vụ đã ghim, không phải từ phần mô tả trách nhiệm của một service.

---

8. Cổng kiểm tra những gì — và không kiểm tra những gì

8.1 Lỗi (cổng fail)

- **Có đủ các mục bắt buộc**: Summary, User Story, Background, Acceptance Criteria, Use cases, Business flow, KB context, Definition of Ready.
- **Không heading nào xuất hiện hai lần.** Một `## Technical grounding` thứ hai — SA viết thêm thay vì thay thế — làm fail cổng; công cụ chỉ đọc bản đầu tiên, nên bản trùng lặp sẽ cho qua một mục đã cũ.
- **Tối đa 10 tiêu chí chấp nhận.** Nhiều hơn mười mục `- [ ]` dưới `## Acceptance Criteria` sẽ làm fail cổng. Hãy tách ticket — gộp hai điều kiện vào một AC cho vừa mức trần chính là điều luật này sinh ra để chặn.
- **Đúng một user story.** Nhiều hơn một dạng `As a … I want … so that …` dưới `## User Story` sẽ làm fail cổng. Mỗi story một ticket.
- **Mọi tham chiếu trong `## KB context` phân giải được** tại commit hub đã ghim. Không có tham chiếu hỏng hay sai định dạng.
- **Mọi trích dẫn `[doc-id §section]` trong nội dung đều có một tham chiếu đã ghim đứng sau.**
- **Change log khớp với revision.** Khi `> Revision: N` lớn hơn 1, `## Change log` phải có đúng một dòng cho mỗi revision `2..N`, theo thứ tự, và mỗi `Trigger` phải là `kb-context`, `architecture`, `scope` hoặc `delivery`.

Trích dẫn viết theo dạng trần kiểu cũ vẫn được tính, kèm cảnh báo nhắc bạn đặt vào ngoặc vuông.

8.2 Cảnh báo (vẫn là phán đoán của bạn)

Cảnh báo	Vì sao không phải lỗi
Một tiêu chí chấp nhận chạm tới tiêu chuẩn nhưng không có trích dẫn	Công cụ không thể xác định đáng tin cậy tiêu chí nào chạm tới tiêu chuẩn
Chất lượng nghiệp vụ của story	Máy không kiểm được
Thiếu dòng `> Revision: <N>`	Nó được đọc là 1, nên file viết trước khi có revision vẫn pass
`## Review record` thiếu, rỗng, hoặc còn nguyên chỗ trống mẫu	Lint thấy được mục đó, không thấy được review có thực chất hay không
Tham chiếu đã cũ	Section được trích đã bị sửa đổi phía trên. Vẫn phân giải được nên không hỏng — nhưng bạn nên xem lại.
Một tham chiếu đã ghim mà nội dung không trích dẫn	Đó có thể là phần nền mà ticket không cần dẫn lại
Một tiêu chí chấp nhận ra lệnh một shell command (`docker compose …`, `curl …`, `grep …`, hoặc hai lệnh nối bằng `&&`)	Công cụ chỉ gắn cờ hình dạng đó; AC có nên đổi sang mô tả kết quả quan sát được hay không là phán đoán của bạn — bản thân câu lệnh thuộc về `## Test data & verification` hoặc kế hoạch
Tiêu chí chấp nhận không ở dạng Given/When/Then (một dòng nêu tên chúng)	Dạng đó là chuẩn; ngưỡng lỗi vẫn là một giá trị đo được hoặc `OPEN(<owner>)` có chủ, nên ticket cũ vẫn pass với một dòng này
Story nhắc tới tải, màn hình, bản ghi hay tích hợp mà mục mở rộng tương ứng đang thiếu	Chỉ là từ khoá trùng; trả lời bằng mục đó hoặc `N/A — <lý do>`
Một mục mở rộng có mặt nhưng rỗng, hoặc `## Sequence diagram` còn là chỗ trống mẫu	Xoá, điền, hoặc viết `N/A — <lý do>`

8.3 Bắt cổng fail khi trích dẫn cũ

```
kb ticket lint tickets/ABC-123.md --fail-on-stale
kb mission lint missions/M-checkout.md --fail-on-stale
kb resolve tickets/ABC-123.md --status-only
```

Hoặc đặt biến repository `KB_FAIL_ON_STALE` để cổng CI làm điều đó cho mọi ticket.

8.4 Vì sao workflow không lọc theo đường dẫn

Workflow được dựng sẵn tên là `kb-ticket-lint` và chạy trên mọi pull request, không chỉ khi có thay đổi dưới `tickets/` hay `missions/`. Đó là chủ đích.

GitHub không bao giờ tự tạo ra trạng thái pass cho một job chưa từng khởi động. Một bộ lọc `paths:` trên một check *bắt buộc* sẽ khiến mọi pull request không đụng tới hai thư mục đó chờ mãi một check không bao giờ chạy. Thay vào đó job luôn khởi động, tự xem diff, và điều phối theo thư mục — thoát sạch kèm thông báo khi không có gì để lint.

Tên workflow cũng có trước cổng mission. Nó được giữ nguyên để tương thích với branch protection; đổi tên sẽ khiến những repository đã bật bảo vệ nhánh bị kẹt ở một check không còn tồn tại.

---

9. Khi một trích dẫn trở nên cũ

Tri thức thay đổi. Một ticket viết hồi tháng Ba dựa trên tài liệu bị sửa vào tháng Năm không được phép âm thầm mang nghĩa khác đi.

```
kb resolve tickets/ABC-123.md          # ok | stale | broken, theo từng tham chiếu
kb diff hr-handbook --against <rev>    # thực sự đã đổi những gì
```

Trạng thái	Nghĩa là gì	Làm gì
`ok`	Nội dung đã ghim vẫn khớp với bản đang publish	Không phải làm gì
`stale`	Nguồn đã bị sửa đổi sau khi bạn trích dẫn	Chạy `kb diff`; quyết định xem tiêu chí chấp nhận có cần cập nhật không
`broken`	Tham chiếu không còn phân giải được	Bám nguồn lại và ghim lại bằng `kb context new` hoặc `kb_context_new`

`kb diff` cho biết chính xác cái gì đã dịch chuyển: tiêu đề section, dòng tóm tắt, phần tóm tắt, bản gốc đầy đủ, section được thêm và bị xoá, và thứ tự tài liệu có đổi hay không.

Phần lớn các trường hợp cũ là vô hại — một lỗi chính tả được sửa ở phía trên. Một số trường hợp là yêu cầu không còn khớp thực tế nữa. Công cụ chỉ cho bạn biết cần xem section nào; phán đoán vẫn là của bạn.

9.1 Kiểm tra hằng ngày

Một ticket đã merge nhiều tuần trước sẽ không bao giờ được cổng của pull request nhìn lại, dù hub vẫn tiếp tục publish. Vì vậy `.github/workflows/kb-doctor.yml` chạy `kb ticket stale` mỗi ngày (và theo yêu cầu) trên `tickets/` và `missions/`. Khi còn bất cứ thứ gì cũ hoặc hỏng, nó giữ **một** issue mở gắn nhãn `kb-stale-citations`; khi mọi thứ đã cập nhật trở lại thì nó đóng issue. Nó cần cùng biến `STRATA_KB_HUB` và secret `KB_HUB_TOKEN` như

cổng lint.

Issue là một bảng có cột `Kind`:

Kind	Nghĩa là	Làm gì
`kb-context`	Một section mà file trích trong `## KB context` đã đổi (`stale`) hoặc không còn phân giải được (`broken`)	Chạy `kb diff` xem đã đổi gì; rồi `/ba-ticket-author --amend` hoặc `/ba-mission-plan --amend` với trigger `kb-context`, lệnh này ghim lại
`architecture`	Một service, bảng, route, tích hợp hay lệnh mà file trích trong `## Services & order` hoặc `## Technical grounding` đã đổi trong tài liệu `-code` của repo Dev kể từ revision nó được ghim (`stale`), hoặc đã bị xoá (`broken`)	Sửa đổi với trigger `architecture`, để `sa-ticket-ground` ghim lại. Kiểm tra các AC dựa trên giá trị đã đổi
`architecture`, lý do `baseline not on hub`	Revision ở `Grounded on:` hoàn toàn không có trên hub: PR của Dev mang nó chưa bao giờ được merge hoặc đã bị thay thế	Đợi `-code` của repo Dev được publish, rồi sửa đổi với trigger `architecture` để ghim chuyển sang một revision hub có

Chỉ các section mà mission hay ticket **trích dẫn** mới được so sánh. Một lần merge của Dev không đụng tới thứ nó dùng thì không tạo dòng nào. Không phải dòng nào cũng cần sửa đổi: hãy đọc thay đổi, và nếu yêu cầu vẫn đúng thì sửa đổi với ghim mới và nói rõ điều đó trong dòng change log.

---

10. Làm việc không cần trợ lý

Mọi thứ agent làm, bạn đều làm được ở terminal.

```
kb query "thời hạn hoàn tiền" --tags payments
kb get payments-spec 4.2 --level l2
kb context new --refs "payments-spec §4.2, payments-spec §4.3"
kb ticket lint tickets/ABC-123.md
kb mission lint missions/M-checkout.md
```

`kb context new` in ra khối trích dẫn; dán nó vào ticket dưới mục `## KB context`.

---

11. Xử lý sự cố

Hiện tượng	Nguyên nhân	Cách xử lý
Lint: "ref does not resolve"	Id section sai, hoặc nội dung đã chưa được publish	Chạy lại `kb query` rồi ghim lại. Kiểm tra PR trên hub đã merge chưa.
Lint: "citation not backed by a pinned ref"	Nội dung trích dẫn thủ mà khối KB context không ghim	Thêm tham chiếu qua `kb_context_new`, hoặc bỏ trích dẫn đó
Lint: "missing required heading"	Thiếu một mục bắt buộc — hoặc mục đó chỉ nằm bên trong khối code	Đưa tiêu đề thật ra ngoài khối code
Lint pass cục bộ, fail trên CI	Chưa đặt `STRATA_KB_HUB` trong Actions variables, hoặc hub đóng tư mà thiếu `KB_HUB_TOKEN`	Cấu hình cả hai; xem mục 2.4
Cổng fail trên PR từ fork	Fork không đọc được secret của repository	Merge qua một nhánh trong chính repository này
Trợ lý không kết nối được hub	`STRATA_KB_HUB_URL` hoặc `STRATA_KB_HTTP_TOKEN` chưa đặt hoặc sai	Chạy lại `kb mcp-setup` (`/kb-mcp-setup` — lệnh này chẩn đoán chính xác cái nào sai, chạy lại không kèm gì rồi xác minh lại mà không cần nhập lại token)
`kb_mcp-setup` báo "token bị từ chối" dù đã có được token mới từ người quản trị hub	Chạy lại không kèm gì chỉ đọc lại đúng token cũ từ `.mcp.json` — lệnh chỉ hỏi lại khi trên đĩa chưa có gì	Chạy `STRATA_KB_HTTP_TOKEN=<token-mới> kb mcp-setup`, hoặc xoá dòng `STRATA_KB_HTTP_TOKEN` trong `.env` rồi chạy lại
`kb query` không tìm thấy gì	Tài liệu chưa được publish, hoặc tag quá hẹp	Bỏ `--tags`; hỏi chủ hub xem PR publish đã merge chưa
`kb publish` không gắn tag và không khởi động CI	`intake:` đang vắng trong `.kb/config.yaml`, nên nó đi đường snapshot	Đặt `intake: {}`
Intake từ chối lần publish của bạn	Repo chưa có trong `federation/registry.yaml` của hub, hoặc mục của nó đặt `ghcm workflow:` (intake sẽ nói với toàn bộ `jobs_workflow_ref` trong đó có tag, nên không bao giờ khớp)	Nhờ người vận hành hub đính ưa `owner/repo` của bạn tới `repo_id` và để nguyên `workflow:`
`kb publish` dừng với " file has uncommitted changes"	Có gì đó dưới `.kb/` khác với `HEAD` — thường là `.kb/index.yaml` hoặc `.kb/<repo_id>/` đang bị theo dõi	Commit phần bạn tự sửa; `git rm --cached` các file do máy sinh rồi chạy lại `kb init --kind ba` để chúng được gitignore
Một mission hay ticket vắng mặt trong plan	Nó không đủ điều kiện	Chạy `kb plan-ingest` và đọc dòng `skipped:` của nó (§ 6.1)
Lint: lỗi "change log" sau khi sửa đổi	`Revision` được dùng mà không có dòng tương ứng, hoặc một `Trigger` viết sai	Thêm dòng một dòng cho mỗi revision (`2...N`, theo thứ tự, dùng bốn giá trị trigger
Sau khi nâng cấp mọi thứ đều `broken`	Ticket mang khối ghim theo định dạng cũ	Ghim lại bằng `kb context new`

12. Tóm tắt lệnh

Lệnh	Mục đích	Mã thoát
`kb mcp-setup [--hub-url URL] [--no-verify]`	Ghi thông tin xác thực HTTP MCP của hub vào `.env` và xác minh chúng	`0` ok, `1` thiếu giá trị hoặc kiểm tra kết nối thất bại
`kb query <text> [--tags t]`	Tìm trong tri thức đã publish	`0`
`kb get <doc> <section>`	Lấy một section	`0`
`kb tags`	Liệt kê từ vựng tag đã publish	`0`
`kb context new --refs "…"`	Sinh khối trích dẫn đã ghim	`0` ok, `1` tham chiếu không phân giải được
`kb resolve <file>`	Kiểm tra các tham chiếu của ticket	`0` tất cả ok, `1` có hỏng, `2` có cũ
`kb diff <doc> --against <rev>`	Sửa đổi đã thay đổi những gì	`0`
`kb ticket lint <file>`	Cổng Definition of Ready cho ticket	`0` PASS, `1` FAIL, `2` cũ
`kb mission lint <file>`	Cổng Definition of Ready cho mission	`0` PASS, `1` FAIL, `2` cũ
`kb ticket check <file>`	Cổng SA grounding: mọi id trong `## Technical grounding` đều có trong `-code` của hub; revision `Grounded on:` bị lệch chỉ fail khi một fact được trích đã mất	`0` PASS, `1` FAIL
`kb ticket export <file>`	Ticket cho hệ thống issue, không có các mục nội bộ của BA	`0`
`kb ticket stale [paths]`	Mọi ticket và mission có trích dẫn mà hub đã dịch chuyển, `kb-context` và `architecture` (`--md`, `--json`)	`0` còn mới, `2` chỉ cũ, `1` hỏng
`kb ticket tidy`	Chuyển `tickets/*.md` phẳng vào `tickets/<mission-id>/` theo dòng cha của chúng	`0`
`kb mission next [--from-hub [--repo id]]`	Story nào done, ready, draft, to-draft hay blocked	`0`
`kb plan-ingest`	Dựng `.kb/<repo_id>-plan/` từ các mission và ticket sẵn sàng; in các dòng `skipped:`	`0`, `1` không chạy được (đầu vào không đọc được, thiếu thư mục, repo id không hợp lệ, đích không do nó sinh, index hỏng, ghi thất bại)
`kb publish`	Sinh lại plan, gắn tag, để CI gửi nó lên hub	`0`, `1` sinh lại plan hoặc build thất bại, hoặc intake từ chối

Slash command: `/ba-ticket-author` (thêm `--amend <ticket>`), `/ba-mission-plan` (thêm `--amend <M-id>`), `/sa-ticket-ground`, `/kb-publish`.

13. Nâng cấp repository

```
kb init --kind ba
```

Lệnh này **chỉ đụng tới file dựng khung** — các CI workflow (gồm `kb-publish.yml`), các skill, file bọc command và prompt, `.gitattributes`, và template dưới `docs/` — và chỉ ghi đè khi nội dung khác với template mới. Nó cũng nối `.kb/*-plan/` và `.kb/index.yaml` vào `.gitignore` nếu còn thiếu, và không bao giờ xoá dòng nào.

`.kb/config.yaml`, `.kb/index.yaml` và `.claude/settings.json` không bao giờ bị `kb init` thường đụng tới. Điều đó gồm cả key `intake:` của `config.yaml`: repo tạo trước khi có publish plan không có key này, nên hãy thêm `intake: <url>` bằng tay (2.5).

> **`--force` thay thế thẳng cả ba**, không hợp nhất: `.kb/config.yaml` bạn đã tự sửa, bao gồm `kind`, `repo_id` và `asset_store`; `.kb/index.yaml`; và toàn bộ `.claude/settings.json` của bạn, bao gồm hooks, permissions và model. Hãy dùng một cách có chủ đích.

Nội dung của riêng bạn dưới `tickets/` và `missions/` không phải là file dựng khung. `kb init` không bao giờ đọc, ghi hay ghi đè bất cứ file nào bạn soạn ở đó — thứ duy nhất nó đặt vào hai thư mục này là một file `.gitkeep` rỗng, để Git theo dõi được thư mục trước khi bạn có ticket đầu tiên.